CMPS 315 - Homework 1: Authentication

Name: Ankit Joshi
UID: C00541037

All logarithms are base 2. Hashes are computed from the exact strings shown, with no trailing newline character.

1. Computing Password Hashes

1(a). MD5 hash of GreatPassword

```
MD5("GreatPassword") = c3cb04673e84c371b2a6876dbf987589
```

1(b). Salt jc3974

The salt is prepended, so the input is the concatenation jc3974GreatPassword.

```
MD5("jc3974GreatPassword") = 69f67e89450e6a1403dbfc3e34f8b4a2
```

1(c). Salt 88p3a9

The input is the concatenation 88p3a9GreatPassword.

```
MD5("88p3a9GreatPassword") = 2927af0d66a0b1477bccc5aba0041a48
```

1(d). Advantages of salting

Without a salt, identical passwords produce identical hashes, as shown in part (a). With a unique salt, the same password produces different stored hashes for different users, as shown in parts (b) and (c). Salting therefore:

- Prevents an attacker from immediately identifying users who share a password.
- Makes one precomputed rainbow table or dictionary-hash list ineffective across all users; the attacker must recompute guesses for each salt.
- Does not make a weak password strong, but it makes large-scale precomputation and bulk cracking much more expensive.

2. Why MD5 and SHA-1 Are Insecure for Password Storage

- They are obsolete cryptographic hash functions with practical collision attacks. Collision resistance is not the only password-storage requirement, but these attacks demonstrate that they are no longer considered modern secure primitives.
- They are fast and inexpensive to compute. That is desirable for general file hashing but dangerous for passwords because attackers can test enormous numbers of guesses per second and can use GPUs or specialized hardware. Password storage should use a slow, memory-hard password hashing function such as Argon2id, or an appropriately configured bcrypt, scrypt, or PBKDF2, with a unique salt.

3. Password Combinations

3(a). Number of possible passwords

There are 65 choices for each position, and passwords may have either 10 or 11 characters:

```
N = 65^10 + 65^11
N = 88,854,106,074,550,781,250 possible passwords
```

The total is 65^10(1 + 65) = 66(65^10).

3(b). Time to test all combinations

At 2,000,000 hashes per second:

```
Time = 88,854,106,074,550,781,250 / 2,000,000 seconds
     = 44,427,053,037,275 seconds
     = 12,340,848,065.91 hours (approximately)
```

3(c). Average time to find the password

Assuming the correct password is equally likely to be anywhere in the search order, the average number of guesses is half the total:

```
Average guesses = N / 2 = 44,427,053,037,275,390,625
Average time = 6,170,424,032.95 hours (approximately)
```

4. Password Entropy

4(a). Ten random printable ASCII characters

```
H = L log2(C) = 10 log2(94) = 65.5459 bits
```

Entropy: approximately 65.55 bits.

4(b). Four random dictionary words

```
H = 4 log2(10,000) = 53.1508 bits
```

Entropy: approximately 53.15 bits.

4(c). Stronger scheme

The 10-random-printable-character scheme is stronger because 65.55 bits is greater than 53.15 bits. Under the stated random-selection assumptions, it has about 2^12.395 (approximately 5,400) times more possible combinations.

5. Full Hash Lookup Table Storage

Each row stores an 8-byte password and a 256-bit SHA-256 hash. Since 256 bits = 32 bytes, each row requires 8 + 32 = 40 bytes.

```
Storage = 10,000,000 rows x 40 bytes/row
        = 400,000,000 bytes
```

Answer: 400,000,000 bytes, ignoring database/index overhead.

6. Number of Rainbow-Table Chains

Each chain represents 5,000 passwords, and the table must represent 10,000,000 passwords:

```
Number of chains = 10,000,000 / 5,000 = 2,000 chains
```

Because the division is exact, no partial chain is needed.

7. Rainbow-Table Size

A rainbow table stores the endpoints of each chain rather than every password/hash pair. For each of 2,000 chains, store one 8-byte starting password and one 32-byte ending SHA-256 hash:

```
Bytes per chain = 8 + 32 = 40 bytes
Total size = 2,000 x 40 = 80,000 bytes
```

Answer: 80,000 bytes, ignoring metadata and indexing overhead.

8. Rainbow Table with One Reduction Function

For each chain, I hashed the current password and reduced the hash to its last five characters. The target hash is checked against every generated hash; when it matches, the current password is the recovered password.

8(a). BartS

Step	Password	MD5 hash
0	world	7d793037a0760186574b0282f2f435e7
1	435e7	c0b33a29a932b715208dcbc227bff4f8 <- target
2	ff4f8	b7b4a61b438038a0eb8b64ff31a35c28
3	35c28	b778d089e7f1cc7a39ef1a1ba039be24
4	9be24	e84939a184d23f6209ae3babdec14d26

The target hash appears at Step 1, so BartS password = 435e7.

8(b). AndyK

Step	Password	MD5 hash
0	frog8	86bf3f38c3b30ae00ab14a055dfe39d7
1	e39d7	1280156f46296baa8cc26406fe3c307d
2	c307d	a82f1ef96c835d01e3bee457593f8482
3	f8482	368c55fc1e92aedad912750149c92a1f
4	92a1f	9b7d09fa6986200fff61a64892d6446d <- target

The target hash appears at Step 4, so AndyK password = 92a1f.

9. Rainbow Table with Three Reduction Functions

I used the reductions cyclically in the stated order: Reduction 1, then Reduction 2, then Reduction 3. The target hash is checked after each MD5 operation.

9(a). JohnD

Step	Password	Operation / MD5 hash
0	hello	MD5 = 5d41402abc4b2a76b9719d911017c592
1	7c592	R1; MD5 = d65782494389c9666b103c5d378d4d9

		c <- target
2	d6578	R2; MD5 = 7b874c84352e16c7a5c0f23aff862860
3	7b860	R3; MD5 = 75481e00bee9b53c1c487e2844d76840

The target hash appears at Step 1, so JohnD password = 7c592.

Final answers:

- BartS: 435e7
- AndyK: 92a1f
- JohnD: 7c592